

StudyShare - Technical Interview Preparation Guide

This guide is designed to help you prepare for technical interviews based on your **StudyShare** project. It covers everything from high-level overviews to deep technical implementations, referencing your exact codebase.

[!TIP]

Use this document to rehearse your answers. Focus on understanding *why* you made certain technical decisions, as interviewers love to ask "Why did you choose X over Y?"

1. Project Overview

What problem does the project solve?

Students often struggle to find reliable, organized study materials for specific subjects and semesters. Materials are usually scattered across WhatsApp groups or Google Drives. Furthermore, many students consume resources without ever contributing back. StudyShare solves this by providing a centralized, searchable repository with a contribution-based access model.

Who are the target users?

College and university students who want to share and access academic resources like notes, past papers, presentations, and assignments.

What is the main idea of the platform?

StudyShare is a collaborative study material sharing platform. Its core defining feature is the **Upload-to-Download Model**: users must contribute (upload) at least one study material before they are granted permission to download materials uploaded by others. This gamifies contribution and ensures the platform's content constantly grows.

2. Complete Working Flow

Step-by-Step Flow:

- Onboarding:** A user visits the platform and signs up. They can either use the traditional Email/Password method (which requires OTP verification via email) or use 1-click Google Login.
- Authentication:** Once verified, the backend generates a JWT (JSON Web Token) which the frontend stores to keep the user logged in.
- Browsing:** The user navigates to the Dashboard where they can search for notes using filters like Subject, Semester, or File Type.
- Upload Requirement:** If a new user tries to download a file, the system checks their `uploadCount`. If it's 0, the download is blocked.
- Contribution:** The user goes to the Upload page and submits a document (e.g., a PDF of their DBMS notes). The file is saved on the server using Multer, and the user's `uploadCount` and `score` increase.
- Access Granted:** Now that `uploadCount > 0`, the user can freely download other materials. (Note: downloading costs 1 score point to prevent spam downloading).

Data Flow Example (Upload):

React UI (FormData) → Axios POST Request → Express Router (/api/notes) → Auth Middleware (Verifies JWT) → Multer Middleware (Saves File to /uploads) → Note Controller (Saves metadata to MongoDB) → Response 201 (Success) back to React.

3. Features Explanation

- Authentication:** Dual support for standard Email/Password (with bcrypt hashing) and Google OAuth integration.
- Email OTP Verification:** Uses Nodemailer to send a 6-digit OTP to verify user emails during signup, preventing spam accounts.
- Google Login:** Allows frictionless onboarding using `@react-oauth/google` and `google-auth-library`.
- File Upload:** Handled via `multer`, restricted to educational formats (PDF, DOC, DOCX, PPT, TXT) with a 10MB size limit.
- File Download:** Serves files directly from the server's file system, tracking download counts for popularity metrics.
- Search and Filters:** Users can query the MongoDB database using Regex to find notes by title, subject, semester, or course.
- User Profile & Leaderboard:** Tracks user activity. Users earn 10 points for uploading and spend 1 point for downloading. Top contributors are displayed on a Leaderboard.
- Upload-to-Download Logic:** A custom Express middleware (`canDownload`) intercepts download requests and checks if `req.user.uploadCount > 0`.
- File Previews:** An endpoint that serves the file inline (instead of as an attachment) so browsers can preview PDFs directly.
- Ratings & Reporting:** Users can rate notes (1-5 stars) and report inappropriate or irrelevant files.

4. Tech Stack Explanation

Frontend

- React.js (v18):** Used to build the Single Page Application (SPA) UI using a component-based architecture. It provides a seamless experience without page reloads.
- Vite:** Used as the build tool instead of Create React App. It provides significantly faster cold starts and Hot Module Replacement (HMR) during development.
- React Router DOM:** Used for client-side routing (e.g., navigating between `/login`, `/dashboard`, `/upload`).
- Axios:** Used to make asynchronous HTTP requests to the Express backend. It automatically parses JSON data and handles headers easily.
- Lucide React:** Used for rendering clean, customizable SVG icons across the UI.

Backend

- Node.js:** The JavaScript runtime environment that executes the backend code.
- Express.js (v5):** The web framework used to quickly build the RESTful APIs, manage routes, and handle middleware.
- MongoDB Atlas & Mongoose:** MongoDB is the NoSQL database chosen for its flexible, document-based structure. Mongoose is the ODM (Object Data Modeling) library used to define strict schemas (`User`, `Note`, `Report`) and interact with the database.
- JWT (jsonwebtoken):** Used for stateless authentication. It securely transmits user ID between the client and server without needing database sessions.
- bcryptjs:** Used to salt and hash plain-text passwords before storing them in MongoDB, ensuring user security in case of a data breach.
- Multer:** A middleware for handling `multipart/form-data`. It intercepts file uploads from the frontend, validates the extension, and saves the physical file to the local disk.
- Nodemailer:** Used to connect to an SMTP server (like Gmail) to programmatically send OTP verification emails.

- **express-rate-limit:** Added to authentication routes to prevent brute-force attacks by limiting IP requests.

5. Folder Structure Explanation

Frontend (/frontend):

- `src/components/`: Reusable UI parts like `Navbar.jsx`, `Footer.jsx`, and `PrivateRoute.jsx`.
- `src/pages/`: Full-page views like `Dashboard.jsx`, `Upload.jsx`, `Login.jsx`.
- `src/context/`: Contains `AuthContext.jsx` for global state management of the user's login session.
- `src/styles/`: Contains standard CSS files for styling the application.
- `App.jsx`: The root component where React Router is configured.

Backend (/backend):

- `server.js`: The entry point. Configures Express, connects to MongoDB, and registers routes.
- `routes/`: Defines API endpoints (`authRoutes.js`, `noteRoutes.js`) and maps them to controllers.
- `controllers/`: Contains the actual business logic (`authController.js`, `noteController.js`).
- `models/`: Defines Mongoose schemas (`User.js`, `Note.js`, `Report.js`).
- `middleware/`: Custom functions that run before controllers (`authMiddleware.js` for JWT verification, `uploadMiddleware.js` for Multer setup).
- `utils/`: Helper functions (`emailService.js` for Nodemailer, `validation.js` for regex checks).
- `uploads/`: The physical directory where user files are stored.

6. Database Design

The database uses MongoDB and consists of three main collections:

1. User Collection (`User.js`):

- *Fields*: name, username, email, password (hashed), isVerified, otp, uploadCount, score.
- *Purpose*: Manages authentication and gamification stats. uploadCount is crucial for the Upload-to-Download logic.

2. Note Collection (`Note.js`):

- *Fields*: title, subject, semester, course, `fileUrl` (filename on disk), `uploadedBy` (ObjectId ref to User), downloadCount, ratings.
- *Purpose*: Stores metadata about uploaded files. The `uploadedBy` field creates a relationship between a Note and a User.

3. Report Collection (`Report.js`):

- *Fields*: note (ObjectId ref), `reportedBy` (ObjectId ref), reason.
- *Purpose*: Allows moderation of inappropriate content.

7. API Explanation

Here are the critical APIs powering the platform:

- `POST /api/auth/signup`: Accepts user details, hashes password, saves unverified user, generates OTP, and triggers email via Nodemailer.
- `POST /api/auth/verify-otp`: Accepts email and OTP. Validates OTP and expiration time. If valid, marks `isVerified = true` and returns a JWT.
- `POST /api/auth/login`: Compares passwords using `bcrypt.compare`. Checks if user is verified. Returns JWT token.
- `POST /api/auth/google`: Accepts a Google OAuth token from the frontend, verifies it using `google-auth-library`, and logs the user in (or creates a pre-verified account).
- `POST /api/notes/`: Protected route. Receives `multipart/form-data`. Multer saves the file. Controller saves metadata and increments User's `uploadCount`.
- `GET /api/notes/`: Fetches notes. Supports query parameters for search, semester, and course filtering using MongoDB Regex.
- `GET /api/notes/download/:id`: Protected route. Intercepted by `canDownload` middleware. If allowed, increments `downloadCount`, decrements User `score`, and triggers `res.download()`.

8. Authentication Flow

Login Flow Deep Dive:

1. User enters Email and Password on frontend.
2. Axios sends `POST /api/auth/login`.
3. Backend finds user by email.
4. Backend uses `bcrypt.compare(req.body.password, user.password)`.
5. If match, it checks `user.isVerified`.
6. If verified, it calls `jwt.sign({ id: user._id }, SECRET)` to create a token.
7. Token is returned to the frontend.
8. Frontend stores token in `localStorage` and includes it as `Authorization: Bearer <token>` in future Axios headers.

9. File Upload System

How Multer Works:

Express cannot parse `multipart/form-data` natively. `uploadMiddleware.js` configures Multer using `diskStorage`.

1. **Destination**: Checks if the `/uploads` folder exists, creates it if not, and sets it as the target.
2. **Filename**: Renames the file to prevent conflicts (e.g., `file-1678901234.pdf`).
3. **Validation**: `fileFilter` checks the extension against a regex (`/pdf|doc|docx|ppt|pptx|txt/`). It also limits size to 10MB.
4. **Database Link**: Only the generated filename (`fileUrl`) is saved in MongoDB. The actual file lives on the server's hard drive.

10. Security Features

- **Password Security**: Plain text passwords are NEVER stored. `bcryptjs` adds a salt and hashes the password.
- **Route Protection**: The `protect` middleware extracts the JWT, verifies it, and attaches the User object to `req.user`. If token is missing/invalid, the API returns 401.

Unauthorized.

- **Brute Force Protection:** `express-rate-limit` prevents spamming the login/signup routes.
- **Input Validation:** Email syntax and password strength are checked in `utils/validation.js` before hitting the database.
- **File Security:** Multer strictly filters file extensions to prevent malicious scripts (like `.exe` or `.sh`) from being uploaded.

11. Deployment Architecture

- **Frontend:** Designed to be deployed on **Vercel** or Netlify. Since it's a Vite app, running `npm run build` generates static HTML/JS/CSS files that Vercel serves via CDN.
- **Backend:** Designed to be deployed on platforms like **Render** or Heroku as a Node web service.
- **Database:** Hosted on **MongoDB Atlas**, a managed cloud database.
- **Environment Variables:** Sensitive data like `MONGO_URI`, `JWT_SECRET`, `EMAIL_PASS`, and `GOOGLE_CLIENT_ID` are kept strictly in `.env` files locally and injected via environment settings in production.

12. Challenges & Solutions

[!NOTE]

Interviewers love hearing about challenges. Use these examples or adapt them to your actual experience.

Challenge 1: Handling File Uploads

- **Problem:** Express doesn't handle files by default, and `req.body` was coming up empty when uploading documents.
- **Solution:** Integrated `multer` to parse multipart form data. Added custom file filtering to ensure only educational documents could be uploaded, preventing server vulnerabilities.

Challenge 2: The Upload-to-Download Logic

- **Problem:** Needed a seamless way to check if a user had permission to download a file without repeating code in every route.
- **Solution:** Created a custom Express middleware (`canDownload`) that runs right after the JWT verification middleware. It checks `req.user.uploadCount` and returns a 403 Forbidden error if it's 0, keeping the controller code clean.

Challenge 3: OTP Expiration

- **Problem:** Initial implementation allowed OTPs to be valid forever, which is a security risk.
- **Solution:** Added an `otpExpiresAt` field to the `User` schema and validated it during the verify step.

13. Future Improvements

- **Cloud Storage Migration:** Currently, files are saved on the local disk (`/uploads`). In a true scalable production environment, files should be uploaded directly to **AWS S3** or **Cloudinary**. This prevents the server from running out of disk space.
- **AI Study Assistant Integration:** Using the Google Gemini API, we could allow users to generate summaries of uploaded PDFs or automatically extract key bullet points for quick revision.
- **Admin Dashboard:** A dedicated interface for moderators to review reported notes and ban malicious users.

14. Interview Preparation Questions

Beginner Questions

Q1: What is the MERN stack?

Answer: MERN stands for MongoDB, Express.js, React, and Node.js. It's a popular tech stack for building full-stack web applications using JavaScript from front to back.

Q2: Why did you use React instead of vanilla HTML/JS?

Answer: React allows for a component-based architecture and uses a Virtual DOM. This makes building a complex Single Page Application like StudyShare much faster and provides a seamless user experience without page reloads.

Q3: What is the purpose of Mongoose in your project?

Answer: Mongoose acts as a bridge between Node.js and MongoDB. It allows me to define strict schemas (like `User` and `Note` models) to ensure data consistency, which MongoDB natively doesn't enforce.

Intermediate Questions

Q4: How does JWT authentication work in your app?

Answer: When a user logs in, the backend creates a JWT containing the user's ID and signs it with a secret key. This token is sent to the React frontend, which stores it. For any protected requests, React sends the token in the Authorization header. The backend verifies the signature, and if valid, processes the request. It's stateless, meaning the server doesn't need to store session data.

Q5: Explain how you implemented the Upload-to-Download logic.

Answer: I added an `uploadCount` field to the `User` schema. Every time a user uploads a file, this count increments. For download routes, I created a custom middleware called `canDownload`. Before the download executes, this middleware checks if `req.user.uploadCount` is greater than 0. If not, it blocks the request with a 403 status.

Q6: Why did you use Multer, and where are files actually stored?

Answer: I used Multer because Express cannot parse `multipart/form-data` requests (which are required for files) out of the box. Currently, Multer saves the physical files to a local `uploads` directory on the server, while the file's generated name and metadata are saved in MongoDB.

Advanced Questions

Q7: Your current file storage uses the local server disk. What is the problem with this in production, and how would you fix it?

Answer: Storing files locally works for development, but in production (like on Heroku or Render), the server's disk is often "ephemeral," meaning files will be deleted when the server restarts. Also, it's hard to scale horizontally across multiple servers. I would fix this by updating my Multer setup to stream uploads directly to a cloud storage bucket like AWS S3 or Cloudinary.

Q8: Walk me through what happens in your backend when a user registers.

Answer: First, the route hits `express-rate-limit` to prevent spam. Then the controller validates the inputs. It checks if the email already exists. If not, it generates a salt and hashes the password using `bcryptjs`. It generates a 6-digit OTP and an expiration time. A new unverified `User` document is saved to MongoDB. Finally, `nodemailer` sends an email to the user with the OTP in the background, and a 201 response is sent to the client.

Q9: How do you prevent users from uploading malicious executable files?

Answer: In my `uploadMiddleware.js`, I implemented a `fileFilter` function. It uses a regular expression `(/pdf|doc|docx|ppt|pptx|txt/)` to check the file extension. If the extension doesn't match these safe document types, Multer throws an error and rejects the upload before it's saved.

Q10: What is the difference between authorization and authentication in your app?

Answer: Authentication is verifying *who* the user is (e.g., checking email/password and issuing a JWT). Authorization is verifying *what* the user is allowed to do (e.g., the `canDownload` middleware checking if the authenticated user has the rights to download a file).

15. Project Summary Pitches

30-Second Pitch (Elevator Pitch):

"StudyShare is a full-stack MERN application that allows college students to share and access academic resources. To ensure continuous growth, I implemented an Upload-to-Download model where users must contribute a file before they can download others. I handled file processing with Multer, authentication with JWT and Google OAuth, and secured the platform with OTP email verification."

1-Minute Pitch (Standard Introduction):

"For my recent project, I built StudyShare, a collaborative MERN stack application. It solves the problem of scattered study materials by providing a centralized, searchable repository for PDFs, PPTs, and notes. The backend is powered by Node, Express, and MongoDB, featuring a robust authentication system that includes standard bcrypt-hashed passwords with Nodemailer OTP verification, as well as Google OAuth. A key feature is the contribution model: a custom middleware ensures users cannot download files until they have uploaded at least one themselves. The frontend is built with React and Vite for a fast, responsive Single Page Application experience. Overall, it gamifies academic sharing through user scores and leaderboards."

3-Minute Pitch (Detailed Technical Overview):

(Combine the 1-minute pitch with details from the Tech Stack, Challenges, and Database Design sections. Focus heavily on how you solved the file upload challenge using Multer and how the `canDownload` middleware operates.)